LoopWear – Internship Project Report (Unified Mentor)

Oct 13, 2026 · @JEET

1. Project details

LoopWear is a Clothing Exchange & Swap Marketplace, built as a Phase 1 internship project for Unified Mentor, where users swap clothes directly instead of buying new ones. It is deployed on Vercel and the code is public on GitHub.

Item	Details
Project title	LoopWear – Sustainable Clothing Exchange & Barter Marketplace
Company	Unified Mentor
Intern name	[Your full name]
Internship domain / role	[e.g. Full Stack Web Development Intern]
Internship duration	[start date] to [end date]
Mentor / guide	[Mentor name]
Live website	https://loopwear-eight.vercel.app
GitHub repository	https://github.com/JeetDodiy/LoopWear
Project type	Full-stack web application (Phase 1, web only)
Package name	`loopwear-clothing-swap`, version 1.0.0

Fill the bracketed fields before you upload; they are details only you know.

2. Abstract and problem statement

Abstract. Fast fashion has increased clothing consumption and textile waste, while many people own wearable clothes they no longer use but find hard to sell or donate. LoopWear is a web marketplace that lets users list unused clothes, browse items from other users, send swap requests, negotiate through chat, and agree on a fair exchange using an estimated swap value. The platform supports location-based matching and an admin

panel for moderation and dispute handling. The system is built with React on the front end, Node.js and Express on the back end, and MongoDB for data, and is deployed on Vercel.

Problem statement. The project brief lists these problems:

- Many people have unused clothes but no easy way to exchange them
- Traditional e-commerce platforms focus only on buying and selling
- Resale platforms can involve complicated pricing
- There are few platforms dedicated to clothing swapping
- Sustainable fashion options are limited for many users

The brief names Depop, Poshmark and Swap.com as reference platforms; LoopWear differs by removing money from the transaction and using direct barter.

3. Objectives and scope

Primary objectives

- Provide a marketplace dedicated to clothing exchange
- Let users swap clothes directly without money changing hands
- Encourage sustainable fashion practices
- Provide location-based swap matching

Secondary objectives

- Enable negotiation between users before a swap is final
- Calculate an estimated swap value
- Filter clothes by category and location
- Offer courier integration for remote swaps (optional)
- Build a community around sustainable fashion

Scope

In scope (Phase 1)	Out of scope (Phase 1)
User registration and login	Online payment system
Clothing listing system	AI-powered fashion recommendations
Swap request system	AR virtual try-on
Negotiation chat between users	Mobile application

In scope (Phase 1)	Out of scope (Phase 1)
Swap value calculator	
Location-based swap suggestions	
Admin management panel	

4. Features and modules

The table lists the modules the project brief requires; I could not open the live app's pages, so tick the status column against your own build before submitting.

Module	Required features	Implemented?
User	Register / login, personal profile, upload listings, manage swap history	[Yes / Partly]
Clothing listing	Image upload; type, size, brand, condition; estimated swap value; availability status; edit or remove	[Yes / Partly]
Swap request	Send request, view incoming requests, accept or reject, track status	[Yes / Partly]
Negotiation chat	Direct messaging, discuss details, confirm agreement	[Yes / Partly]
Swap value calculator	Value from brand, condition and category; fair-match suggestions; value comparison of both items	[Yes / Partly]
Location matching	Nearby users, location filter, nearby swap suggestions	[Yes / Partly]
Admin panel	Manage users and listings, monitor swaps, remove inappropriate listings, resolve disputes, analytics	[Yes / Partly]

Pages required by the brief

The brief asks for 6–8 interconnected pages: Login, Clothing Listings, Item Detail, Swap Request, Chat, User Dashboard and Admin Panel.

Swap flow

1. User registers or logs in and creates a profile

2. User uploads a clothing listing with details and estimated value
3. User browses other listings by category and location
4. User sends a swap request for an item
5. Both users negotiate in chat and confirm the agreement
6. Items are exchanged locally or shipped by courier
7. Admin monitors activity and resolves disputes

5. Technology stack

LoopWear is a JavaScript full-stack app: a React client, an Express API and MongoDB, hosted on Vercel. Back-end packages below come from the repository's package.json; the front-end details are from the project brief and your code layout, so check them against the `client/package.json` file.

Layer	Technology	Purpose
Front end	React.js, HTML5, CSS3, JavaScript (Vite dev server on port 5173, to confirm)	Pages, forms, chat and dashboards
Styling	Bootstrap or Tailwind CSS (confirm which one)	Responsive, mobile-friendly layout
Back end	Node.js with Express 4 (ES modules)	REST API
Database	MongoDB with Mongoose 8	Users, listings, swaps, chats, disputes
Password security	bcryptjs	Hash passwords
Authentication	jsonwebtoken (JWT)	Signed login tokens with expiry
Security headers	Helmet	CSP, HSTS, X-Frame-Options and more
Abuse protection	express-rate-limit	Limit login and API attempts
Cross-origin rules	cors	Allow only the app's own origin
Configuration	dotenv	Keep secrets in environment variables

Layer	Technology	Purpose
Hosting	Vercel (`vercel.json` rewrites, `api/` folder)	HTTPS deployment of client and API
Version control	Git and GitHub (10 commits on `main`)	Source history

Why these choices

- **React** gives reusable components for listing cards, chat and dashboards.
- **Express and MongoDB** keep the API simple, and documents suit clothing items with varying details.
- **Vercel** gives free HTTPS and quick deployment, which matches the brief's requirement for a live link.

6. Project structure and security

The repository root holds the client, the server and the Vercel API entry, plus design and planning files.

```
LoopWear/
  client/              React front end (build: npm run build)
  server/              Express API (start: npm start)
  api/                 Vercel serverless entry for the API
  vercel.json          Rewrites and deployment config
  package.json         Root scripts and server dependencies
  context.txt          Project brief from Unified Mentor
  validation.txt       Security and validation review notes
  color-plate.png      Brand color palette
  logo.png             LoopWear logo
  README.md
```

Security and validation measures

The repository's validation notes list problems found in an earlier version (hardcoded passwords, plaintext storage, unprotected admin routes, open CORS, no rate limiting). The current dependencies show the fixes were started: bcryptjs, jsonwebtoken, helmet and express-rate-limit are all installed. Confirm each fix in `server.js` before you write it as done.

Area	Measure	Status
Passwords	Hash with bcryptjs; never return password fields	[Done / Confirm]
Authentication	Signed JWT with expiry and role claim	[Done / Confirm]
Authorization	Auth middleware on all routes; admin-only role check on `/api/admin/*`	[Done / Confirm]
Headers	Helmet enabled	[Done / Confirm]
Rate limiting	Limit auth attempts (for example 10 per 15 minutes)	[Done / Confirm]
CORS	Restrict to the Vercel site and localhost	[Done / Confirm]
Body size	JSON limit 2 MB instead of 20 MB	[Done / Confirm]
Database	MongoDB required in production; TLS certificate validation on	[Done / Confirm]
Input validation	Schema validation (Zod or express-validator), duplicate-email check, enum checks for status and role	[Done / Confirm]
IDs	`crypto.randomUUID()` instead of timestamp IDs	[Done / Confirm]

7. Deployment, learnings and conclusion

Deployment

The app is deployed on Vercel at loopwear-eight.vercel.app. The root `build` script builds the client (`cd client && npm run build`) and `start` runs the server (`cd server && npm start`). Production needs a `MONGODB_URI` environment variable, because Vercel functions are ephemeral and an in-memory store loses data on every restart.

Key performance indicators

KPI	Meaning
Number of clothing listings	Items uploaded by users
Successful swaps	Swap requests that reached an accepted, completed state
Swap request conversion rate	Accepted requests ÷ sent requests × 100

KPI	Meaning
User engagement rate	Users who list, request or chat ÷ registered users × 100
Active users	Users with activity in the last 30 days

Challenges and how they were solved

Challenge	Solution
Fair swaps between different items	Swap value calculator based on brand, condition and category, plus chat negotiation
Data lost on serverless hosting	Move to MongoDB as the production database
Insecure first version (found in review)	Add password hashing, JWT, Helmet, rate limiting, CORS rules and validation
Matching users by place	Location field on users and listings with location filter and nearby suggestions

Skills learned

- Building a full-stack app with React, Express and MongoDB
- Designing REST APIs, roles and a swap workflow with statuses
- Securing an app: hashing, JWT, rate limiting, validation, headers
- Deploying to Vercel and managing environment variables
- Using Git and GitHub, and reviewing code against a checklist

Future scope

- AI-based swap recommendations
- Mobile application
- Clothing condition verification
- Sustainability impact tracker (waste saved)
- Community fashion groups
- Courier integration and online payments for shipping costs

Conclusion

LoopWear delivers a working, deployed clothing swap marketplace with listings, swap requests, negotiation chat, value estimates, location matching and an admin panel, and it supports the goals of reusing clothes and cutting textile waste. Future phases can add recommendations, a mobile app and an impact tracker.

Acknowledgement

I thank Unified Mentor and my mentor [Mentor name] for the opportunity and guidance during this internship.